

بسمه تعالی

پروژه درس برنامه نویسی پیشرفته

دانشکده فنی انقلاب اسلامی

دانشگاه ملی مهارت

استاد درس: دکتر اردستانی

نام و نام خانوادگی دانشجو: امیر پارسا علیزاده

شماره دانشجویی: 02220040709015

نکته: در تمام این پروژه ها a آخرین رقم غیر صفر شماره دانشجویی شماست.

گزارش تحلیل کد: نرم افزار تحلیل پاسخ سیستم های مرتبه اول و دوم

۱. معرفی کلی پروژه

این برنامه یک ابزار آموزشی کنترل با رابط گرافیکی (GUI) است که با گرفتن پارامترهای ω_n ، K و در صورت نیاز ζ ، تابع تبدیل یک سیستم مرتبه اول یا دوم را می سازد، پاسخ پله ی آن را شبیه سازی می کند و مشخصه های پاسخ گذرا (زمان صعود، زمان نشست، فراجش) را روی نمودار و در قالب عددی نمایش می دهد.

۲. دسته بندی بخش های کد

بخش	کلاس/محدوده	نقش
ساخت مدل ریاضی سیستم	TransferFunction	ساخت و اعتبارسنجی تابع تبدیل
شبیه سازی دینامیک	StepResponse	محاسبه ی پاسخ پله در طول زمان
ساختار داده نتیجه	AnalysisResult (dataclass)	نگهداری خروجی های تحلیل به صورت یک جا
استخراج مشخصات	Analyzer	محاسبه ی زمان صعود، نشست، فراجش و پیک
رسم و نمایش	Plotter	ترسیم نمودار پاسخ پله و برجسب گذاری آن
رابط کاربری	ControlAnalyzerGUI	دریافت ورودی، اجرای زنجیره ی تحلیل، نمایش نتیجه

۳. تحلیل کلاس TransferFunction

۳.۱. اعتبارسنجی متمرکز ورودی ها

```
@staticmethod
def _validate_inputs(order, K, wn, zeta):
    if order not in (1, 2): raise ValueError(...)
    if wn <= 0: raise ValueError(...)
    if K == 0: raise ValueError(...)
    if order == 2:
        if zeta is None: raise ValueError(...)
        if zeta < 0: raise ValueError(...)
```

اعتبارسنجی به جای پخش شدن در چند نقطه از برنامه، در یک متد `staticmethod` واحد متمرکز شده است. طراحی به صورت `staticmethod` نشان می دهد این منطق به هیچ نمونه ی خاصی از کلاس وابسته نیست و صرفاً روی مقادیر ورودی عمل می کند — این کار خطا را در همان لحظه ی ساخت آبجکت (Fail Fast) گزارش می کند، نه در وسط شبیه سازی که ردیابی علت آن دشوارتر است.

۳.۲. ساخت شرطی ضرایب بر اساس مرتبه

```
if self.order == 1:
    num = [self.K * self.wn]
    den = [1, self.wn]
else:
    num = [self.K * self.wn ** 2]
    den = [1, 2 * self.zeta * self.wn, self.wn ** 2]
```

برای مرتبه اول از فرم استاندارد $G(s) = K \cdot \omega_n / (s + \omega_n)$ استفاده شده که معادل $K/(\tau s + 1)$ با $\tau = 1/\omega_n$ است. استفاده‌ی مستقیم از ω_n به جای تبدیل آن به ثابت زمانی τ ، هم از خطای تقسیم و هم از افت دقت عددی جلوگیری می‌کند.

۳.۳. متد describe

این متد مسئولیت تولید نمایش متنی تابع تبدیل را از منطق رسم جدا کرده است؛ نمونه‌ای از جداسازی داده از نمایش (Separation of Data and Presentation)، مشابه الگویی که در کلاس‌های دیگر مثل نگهداری رنگ‌ها در TrafficLight دیده می‌شود.

۴. تحلیل کلاس StepResponse

۴.۱. تخمین هوشمند بازه‌ی زمانی شبیه‌سازی

```
def _estimate_t_final(self):
    wn = self.tf.wn
    zeta = self.tf.zeta if self.tf.order == 2 else 1.0
    zeta = max(zeta, 0.05)
    t_final = 8 / (zeta * wn)
    return max(t_final, 1.0)
```

این متد:

- برای سیستم مرتبه اول که $zeta$ تعریف نشده است، مقدار 1.0 جایگزین می‌شود تا فرمول زمان نشست (که بر پایه‌ی رفتار میراشده طراحی شده) بدون شرط جداگانه قابل استفاده باشد.
- $\max(zeta, 0.05)$ از تقسیم بر عددی نزدیک صفر و در نتیجه بازه‌ی زمانی بی‌اندازه بزرگ (برای سیستم‌های تقریباً بدون میرایی) جلوگیری می‌کند.
- $\max(t_final, 1.0)$ یک کف زمانی تضمین می‌کند تا برای ω_n های بزرگ (سیستم‌های خیلی سریع)، بازه‌ی نمایش بیش‌ازحد کوتاه و غیرقابل خوانش نشود.

این الگو نمونه‌ای از تخمین پارامتر شبیه‌سازی بر اساس ویژگی‌های فیزیکی خودِ سیستم است، به جای گرفتن یک مقدار ثابت یا از کاربر.

۵. تحلیل کلاس — Analyzer

۵.۱. ساختار خروجی یکپارچه با AnalysisResult

```
@dataclass
class AnalysisResult:
    final_value: float
    rise_time: Optional[float]
    settling_time: Optional[float]
    overshoot_percent: float
    peak_value: float
    peak_time: Optional[float]
```

به جای بازگرداندن یک دیکشنری یا چند مقدار جدا، از dataclass استفاده شده است. این کار دسترسی به فیلدها را با نقطه (result.settling_time) به جای کلید رشته‌ای ممکن می‌کند، از خطای تایپی در نام کلید جلوگیری می‌کند و به Plotter و GUI یک قرارداد (Contract) روشن برای شکل داده‌ی ورودی می‌دهد.

۵.۲. مدیریت حالت‌های نامعتبر با Optional

```
def _compute_rise_time(self, final_value):
    low, high = 0.1 * final_value, 0.9 * final_value
    try:
        t_low = self.t[np.where(self.y >= low)[0][0]]
        t_high = self.t[np.where(self.y >= high)[0][0]]
        return float(t_high - t_low)
    except IndexError:
        return None
```

اگر خروجی سیستم هرگز به ۹۰٪ مقدار نهایی نرسد (مثلاً بازه‌ی زمانی کوتاه بوده یا سیستم بسیار کند است)، np.where(...) [0][0] با IndexError مواجه می‌شود. این خطا به جای متوقف کردن برنامه، به None تبدیل می‌شود — الگویی که در سطح GUI نیز با نمایش "—" ادامه پیدا می‌کند. این زنجیره‌ی مدیریت خطا (از محاسبه تا نمایش) بدون هیچ try/except در لایه‌های بالاتر کار می‌کند.

۵.۳. تشخیص زمان نشست با پیمایش معکوس باند خطا

```
def _compute_settling_time(self, final_value):
    band = self.tol * abs(final_value)
    outside_band = np.where(np.abs(self.y - final_value) > band)[0]

    if len(outside_band) == 0:
        return float(self.t[0])

    last_outside_index = outside_band[-1]
    if last_outside_index + 1 >= len(self.t):
```

```

return None
return float(self.t[last_outside_index + 1])

```

منطق این متد را می‌توان این‌طور خلاصه کرد: «تمام نقاطی که بیرون از باند ۲٪ هستند را پیدا کن، آخرین آن‌ها را بیاب، و زمان نشست همان لحظه‌ی بعدی است چون از آن نقطه به بعد سیگنال دیگر هرگز از باند خارج نمی‌شود»

دو حالت مرزی به‌شکل صریح مدیریت شده‌اند:

- اگر هیچ نقطه‌ای بیرون باند نباشد (سیگنال از ابتدا داخل بازه‌ی مجاز است)، اولین لحظه‌ی زمانی برگردانده می‌شود.
- اگر آخرین نقطه‌ی بیرون‌ازباند، آخرین نمونه‌ی کل شبیه‌سازی باشد (یعنی سیستم تا پایان بازه هم مستقر نشده)، None برگردانده می‌شود؛ این علامتی است که بازه‌ی زمانی تخمینی در StepResponse برای این سیستم خاص کافی نبوده است.

۵.۴. جلوگیری از فراجهبش منفی

```

def _compute_overshoot(self, final_value, peak_value):
    if final_value == 0:
        return 0.0
    overshoot = (peak_value - final_value) / final_value * 100
    return max(overshoot, 0.0)

```

در سیستم‌های بیش‌میرا (overdamped) یا در حضور خطای عددی جزئی، ممکن است peak_value اندکی کمتر از final_value محاسبه شود که به فراجهبش منفی و بی‌معنی منجر می‌شود. عبارت max(overshoot, 0.0) این حالت را در همان لحظه‌ی محاسبه خنثی می‌کند، به‌جای آنکه به لایه‌ی نمایش منتقل شود.

۶. تحلیل کلاس Plotter

۶.۱. بازاستفاده از یک Figure واحد

```

def __init__(self, figsize=(7, 5)):
    self.fig, self.ax = plt.subplots(figsize=figsize)

def plot(self, step_response, result, tf):
    self.ax.clear()
    ...

```

برخلاف رویکردی که در هر فراخوانی یک Figure جدید بسازد، اینجا fig و ax فقط یک‌بار در __init__ ساخته می‌شوند و در هر رسم با ax.clear() پاک و دوباره پر می‌شوند. این طراحی باعث می‌شود در طرف GUI نیازی به ساخت FigureCanvasTkAgg جدید در هر بار کلیک روی دکمه نباشد؛ کارآمدتر است و از تجمع اشیای گرافیکی بلااستفاده در حافظه جلوگیری می‌کند.

۶.۲. نمایش شرطی برجسب‌های تحلیلی

```
if result.overshoot_percent > 0.01:
```

```
    self.ax.plot(result.peak_time, result.peak_value, "ro")
```

```
    self.ax.annotate(f'Overshoot = {result.overshoot_percent:.2f}%', ...)
```

annotation فراجاهش فقط زمانی رسم می‌شود که مقدار آن از یک آستانه‌ی بسیار کوچک (۰.۰۱٪) بیشتر باشد. این شرط، سیستم‌های overdamped یا critically damped (که عملاً فراجاهش ندارند) را از شلوغ شدن نمودار با برجسب‌های بی‌معنی مصون نگه می‌دارد.

۶.۳. تفکیک منطق نمایش زمان نشست و زمان صعود

هرکدام از settling_time و rise_time پیش از رسم با `if result.xxx is not None` بررسی می‌شوند؛ یعنی همان مقدار `None` که در Analyzer تولید شده، مستقیماً تصمیم می‌گیرد که آیا خط یا برجسب مربوطه اصلاً رسم شود یا نه — زنجیره‌ی داده تا لایه‌ی نمایش بدون تبدیل اضافی جریان دارد.

۷. تحلیل کلاس ControlAnalyzerGUI

۷.۱. غیرفعال‌سازی پویای فیلد zeta

```
def _toggle_zeta_field(self):
```

```
    if self.order_var.get() == 1:
```

```
        self.zeta_entry.configure(state="disabled")
```

```
    else:
```

```
        self.zeta_entry.configure(state="normal")
```

این متد به رادیوباتن‌های انتخاب مرتبه متصل است (`command=self._toggle_zeta_field`) و باعث می‌شود فیلد zeta فقط زمانی قابل ویرایش باشد که برای مدل معنا داشته باشد. این کار از ورود مقدار نامعتبر یا گیج شدن کاربر روی مرتبه اول جلوگیری می‌کند و هم‌زمان در `_on_run` با شرط `zeta = float(...) if order == 2 else None` تکمیل می‌شود.

۷.۲. زنجیره‌ی اجرای خطی در `_on_run`

```
tf = TransferFunction(order=order, K=K, wn=wn, zeta=zeta)
```

```
step_resp = StepResponse(tf)
```

```
result = Analyzer(step_resp).analyze()
```

```
self.plotter.plot(step_resp, result, tf)
```

```
self.canvas.draw()
```

```
self._update_result_labels(result)
```

این متد به روشنی مسیر داده را در چهار گام نشان می‌دهد: مدل‌سازی ← شبیه‌سازی ← تحلیل ← نمایش. هر کلاس فقط خروجی کلاس قبلی را می‌گیرد و به کلاس بعدی می‌دهد، بدون آنکه از جزئیات داخلی آن باخبر باشد — نمونه‌ای از معماری پایپ‌لاین (Pipeline Architecture) در مقیاس کوچک.

۷.۳. مدیریت دوسطحی خطا

```
except ValueError as e:
    ", str(e)) خطای ورودی" messagebox.showerror("
except Exception as e:
    ":\\n{e}" "مشکلی پیش آمد" f"خطا" messagebox.showerror("
```

خطاهای اعتبارسنجی `ValueError`، که عمده‌اً در `TransferFunction._validate_inputs` تولید می‌شوند از خطاهای پیش‌بینی‌نشده `Exception` عمومی تفکیک شده‌اند. این کار به کاربر پیام دقیق‌تر و مرتبط‌تری